

Problem-Solution Fit

Date	29 September 2026
Team ID	SWTID-2026-1307
Project Name	Script-Controlled ACL-Restrict Record Access Based on Field Value.
Maximum Marks	5 Marks

Problem-Solution Fit Canvas:

1. CUSTOMER SEGMENT(S) [CS] - ServiceNow administrators and developers who own security and access design. - Security / compliance officers responsible for audits (data privacy, internal policies). - Service desk agents and fulfillers (ITIL role) who work daily on incident, HR and request records. - Department managers (HR, Finance, Legal) whose records must stay confidential.	6. CUSTOMER LIMITATIONS EG. BUDGET, DEVICES [CL] - Low budget: must use only out-of-the-box ServiceNow features, no paid plugins. - Development on a free Personal Developer Instance (PDI) with limited resources. - Only a web browser is needed; no special device or software. - Many admins have limited scripting (GlideRecord / server-side JS) knowledge. - Rules must not slow down list and form loading.	7. BEHAVIOR + ITS INTENSITY [BE] - Users open records from lists, forms, reports, global search and portal many times every day (high). - Users click or search records outside their duty, by curiosity or mistake (medium). - Admins create extra roles or hide fields as a quick fix each time a new need comes (medium). - Security reviews and audits happen periodically (low, but high impact).
2. PROBLEMS / PAINS + ITS FREQUENCY [PR] - Records with sensitive field values (e.g. Confidential = true, Department = HR) are visible to every user having table-level role (daily). - Role-based ACL cannot check the value inside a record (every record access). - Hiding fields by UI policy / client script is bypassed through list view, reports and API (frequent). - Audit observations for unauthorized access (every quarter / audit cycle).	9. PROBLEM ROOT / CAUSE [RC] - Default ACLs check only role or table, not the field value of the record being opened. - No dynamic check of the record against the user (owner, assignment group, department). - Client-side controls are wrongly treated as real security, but they are not enforced on the server. - Access design is done late, and scripted ACL knowledge is low.	8. CHANNELS OF BEHAVIOR [CH] ONLINE - ServiceNow web UI: lists, forms, dashboards, global search. - Service Portal / mobile app. - REST / Table API and reports.
3. TRIGGERS TO ACT [TR] - A confidential record is seen by an unauthorized user (data leak). - Audit or compliance requirement to restrict access. - New confidential process (HR cases, payroll, legal) is onboarded. - Complaint from a manager or record owner. 4. EMOTIONS BEFORE / AFTER [EM] Before: worried, exposed, frustrated, afraid of audit failure, low trust in the system. After: confident, secure, in control, relieved and audit-ready.	10. YOUR SOLUTION [SL] - Create a script-controlled ACL (type: record, operation: read / write) on the target table. - The ACL script reads current.<field> and compares it with the logged-in user (role, ID, group, department). - Only matching users (owner, assigned group, authorized role, admin) get access; all others are denied. - Enforced on the server side, so it works in forms, lists, reports, search and API. - Verified by impersonating users with different roles. <pre> answer = false; if (current.u_sensitivity != 'confidential') answer = true; else if (gs.hasRole('confidential_reader') current.assigned_to == gs.getUserID()) answer = true; </pre>	8. CHANNELS OF BEHAVIOR [CH] OFFLINE - Phone calls / walk-ins to the service desk asking for records. - Emails, meetings and verbal requests to admins for access. - Printed reports and spreadsheets shared outside the system.

5. AVAILABLE SOLUTIONS

PROS & CONS [AS]

Available solution	Pros	Cons
Role-based (table-level) ACL	Out of the box, easy to set up.	Cannot restrict by field value; all role holders see all records.
Condition-based ACL (no script)	No coding, quick to configure.	Only simple conditions; cannot compare record with user's group / department or use logic.
UI policy / client script / Business rule to hide data	Fast to build, easy to change.	Client-side, bypassed via list, report or API; not real security.
Domain separation / separate tables	Strong data isolation.	Costly, complex, heavy for a small requirement.
Manual control (email approvals, spreadsheets)	No development needed.	Slow, error-prone, no audit trail.